

Функция EFF_k^G

Дубровин Никита, 23.Б07-мм

github.com/uwul3n/formal-languages

Постановка задачи

Для контекстно-свободной грамматики $G = (V_N, V_T, P, S)$ и параметра $k \geq 1$ требуется вычислить функцию $\text{EFF}_k^{G(\alpha)}$, где $\alpha \in (V_N \cup V_T)^*$:

$$\text{EFF}_k^{G(\alpha)} = \begin{cases} \text{FIRST}_k^{G(\alpha)}, & \text{если } \alpha \text{ начинается на терминал} \\ \left\{ w \in V_T^{*k} \mid \exists \alpha \xRightarrow[\text{rm}]{*} \beta \xRightarrow[\text{rm}]{} wx, \beta \neq Awx \right\}, & \text{иначе} \end{cases}$$

то есть $\text{EFF}_k^{G(\alpha)}$ исключает терминальные цепочки, у которых последний шаг правостороннего вывода имеет вид $A \rightarrow \varepsilon$ в позиции $\beta = Awx$ (ε -правило на ведущем нетерминале после полного раскрытия хвоста).

Решение

Развернём определение в рекурсивную процедуру. Если $\alpha = a\beta$ (терминал в начале), $\text{EFF}_k = \text{FIRST}_k$ непосредственно. Если $\alpha = A\beta$, то в правостороннем выводе хвост β полностью раскрывается до терминалов раньше, чем затрагивается A ; значит, последний шаг — это либо $A \rightarrow \varepsilon$ (запрещено определением), либо потомки A в подвыводе из $A \rightarrow \gamma$, $\gamma \neq \varepsilon$. В первом случае результат отбрасываем, во втором продолжаем тот же анализ для строки $\gamma\beta$:

$$\text{EFF}_k^{G(A\beta)} = \bigcup_{A \rightarrow \gamma, \gamma \neq \varepsilon} \text{EFF}_k^{G(\gamma\beta)}.$$

Рекурсия может заикливаться (например, $A \rightarrow B$, $B \rightarrow A$); решается обходом в ширину по строкам с множеством `visited`.

FIRST_k^G для нетерминалов считается заранее как наименьшая неподвижная точка: для каждого правила $A \rightarrow \gamma$ добавляем $\text{FIRST}_k^{G(\gamma)}$ в $\text{FIRST}_k^{G(A)}$ до стабилизации. Конкатенация с k -усечением — стандартная операция $\oplus_k$.

Псевдокод

```
function FirstK( $\alpha$ , k):
  R <- { $\varepsilon$ }
  for X in  $\alpha$ :
    F <- {X} если X терминал, иначе FIRST_k[X]
    R <- R  $\oplus_k$  F
    если все строки в R длины k: break
  return R

function EffK( $\alpha$ , k):
  если  $\alpha$  пуста: return  $\emptyset$ 
  если  $\alpha[0]$  терминал: return FirstK( $\alpha$ , k)
  queue <- [ $\alpha$ ]; visited <- { $\alpha$ }; result <-  $\emptyset$ 
  while queue непуста:
    form <- pop queue
    если form[0] терминал:
      result <- result  $\cup$  FirstK(form, k); continue
```

```

A <- form[0]; tail <- form[1:]
for каждое правило A -> γ, γ ≠ ε:
    next <- γ ++ tail
    если next ∉ visited: visited.add(next); queue.push(next)
return result

```

Сложность

Пусть $|G|$ — суммарная длина правил, $N = |V_N|$, $|V_T| = t$. Подсчёт FIRST_k^G — стандартный, $O(|G| \cdot |V_T|^k \cdot N)$ в худшем случае. $\text{EFF}_k^{G(\alpha)}$ — BFS по строкам; для грамматик без бесполезных циклов число посещаемых строк ограничено и обход завершается за полиномиальное время от $|G|$ и k .

Формат входа

```

# терминалы
a b c
# нетерминалы
S A B
# стартовый символ
S
# количество правил
5
# правила: LHS -> X1 X2 ...; ε записывается как eps
S -> A B
A -> a
A -> eps
B -> b
B -> c
# k
1
# запросы (по одному α на строку)
S
A B
A
B
a B

```

Пример

Для приведённой грамматики и $k = 1$:

```

EFF_1(S) = {a}
EFF_1(A B) = {a}
EFF_1(A) = {a}
EFF_1(B) = {b, c}
EFF_1(a B) = {a}

```

Из $\text{EFF}_1(S)$ исключены **b** и **c**: они достижимы только выводами $S \Rightarrow AB \Rightarrow_{\text{rm}} Ab \Rightarrow_{\text{rm}} b$ и $S \Rightarrow AB \Rightarrow_{\text{rm}} Ac \Rightarrow_{\text{rm}} c$, где последний шаг — $A \rightarrow \varepsilon$ при $\beta = Awx$.

Для грамматики арифметических выражений (`example2.txt`, $E \rightarrow TE_1$, $E_1 \rightarrow +TE_1 \mid \varepsilon$, $T \rightarrow FT_1$, $T_1 \rightarrow *FT_1 \mid \varepsilon$, $F \rightarrow (E) \mid \text{id}$) и $k = 1$:

```
EFF_1(E)    = {(, id}
EFF_1(E1)   = {+}
EFF_1(T1)   = {*}
EFF_1(F)    = {(, id}
EFF_1(T E1) = {(, id}
```

В $\text{EFF}_1(E_1)$ нет ε (хотя $\varepsilon \in \text{FIRST}_1(E_1)$): единственный способ получить пустой префикс — применить $E_1 \rightarrow \varepsilon$, что и есть запрещённый последний шаг.

Сборка и запуск

```
cd build && cmake .. && cmake --build .
./eff_app ../example.txt
./eff_app ../example2.txt
```